Revised 120-Day Career Switch Roadmap

> From Enterprise / Production Experience to AI-Powered Backend & GenAI Engineering

Timeline: 120 days / approximately 17 weeks

Core Direction: Python + Backend + GenAI + Cloud + System Design + DSA

Target: AI-powered backend / GenAI engineering roles

> **Core positioning:** Software Engineer with enterprise production experience who builds AI-powered systems.

1. Why This Is a 120-Day Plan

The original 90-day roadmap was ambitious for someone working a demanding full-time job. The goal of this revision is not to add unnecessary technologies. The extra 30 days are used for better pacing, repetition, project quality and interview preparation.

The strategy remains the same: build on enterprise production experience and transition toward Python backend engineering and AI-powered application engineering.

The biggest advantage of the 120-day version: consistency is more important than hitting aggressive daily study targets.

2. Realistic Weekly Schedule

Weekdays: approximately 1.5–2 hours per day.

Recommended weekday structure: 30–45 minutes DSA + 45–60 minutes learning or project work.

Saturday: approximately 3–4 hours — project work, DSA and revision.

Sunday: approximately 2–3 hours — lighter project work, revision, career work or certification.

Weekly target: approximately 12–16 focused hours.

Busy-day minimum: 45–60 minutes. A smaller session is better than abandoning the day.

Weekly philosophy: Hit the weekly minimum and carry unfinished topics forward. Do not treat a difficult work week as failure.

3. Priority System

1. Projects and real coding: highest priority.

2. Python + backend engineering: highest priority.

3. GenAI application engineering: highest priority.

4. DSA: pattern recognition and interview confidence.

5. Cloud: practical AWS knowledge and deployment.

6. Certifications: useful but secondary. Never let certificates replace projects or coding.

PHASE 1 — DAYS 1–28

Focus: Python + DSA Foundations.

Main objective: Become comfortable translating an idea into working Python code before accelerating into backend and advanced DSA topics.

This phase intentionally gives you four weeks rather than rushing the foundation into three.

WEEK 1 — Python Fundamentals + Arrays

Python: variables, data types, lists, tuples, loops, conditionals and functions.

DSA theory: algorithms, time complexity, space complexity, O(1), O(n), O(n^2) and O(log n).

Practice: array traversal, max/min, counting, simple manipulation and basic frequency problems.

Target: approximately 10–15 quality problems.

WEEK 2 — Strings + Hashmaps + Sets

Python: string operations, indexing, slicing, iteration and useful methods.

Data structures: dictionaries, .get(), dictionary iteration, sets and membership lookup.

DSA: frequency counting, duplicate detection, Two Sum-style problems, intersections and missing-element patterns.

Target: approximately 20–25 total problems.

WEEK 3 — OOP + Two Pointers + Sliding Window

Python OOP: classes, objects, __init__, instance variables, methods, class variables, @classmethod and @staticmethod.

Coding exercise: build a Ticket class with ticket ID, severity, status and service.

DSA: two pointers, opposite-direction pointers, same-direction pointers and sliding window.

Target: approximately 30–35 total problems.

WEEK 4 — Stack + Linked Lists + Binary Search

Python: exceptions, modules and useful collections such as deque, defaultdict and Counter.

DSA: stack, queue basics, linked list fundamentals, reverse linked list, middle node, fast/slow pointers and binary search.

Recursion: base case, recursive case and call stack.

End-of-Phase target: approximately 40–45 quality problems.

Checkpoint: you should be comfortable with arrays, strings, hashing, two pointers, sliding window, stack basics, linked lists and binary search.

PHASE 2 — DAYS 29–49: Backend Foundations

Focus: HTTP, REST, FastAPI, PostgreSQL and production-oriented backend development.

WEEK 5 — HTTP + REST + API Fundamentals

Learn HTTP methods, REST, JSON, status codes, request/response lifecycle, validation, authentication concepts and pagination.

Begin FastAPI: routes, request models, response models and Pydantic.

DSA: review previous topics and begin tree fundamentals.

WEEK 6 — FastAPI + Production Patterns

Learn dependency injection, middleware, exception handling, logging, async endpoints and OpenAPI.

Start **Project #1: Production Ticket Management API**.

Features: create/update/retrieve tickets, validation, authentication and logging.

DSA: trees, traversals and BST basics.

WEEK 7 — PostgreSQL + Docker

Learn PostgreSQL tables, joins, indexes, transactions, ACID, normalization, isolation and query plans.

Learn Docker images, containers, volumes, networks, Dockerfiles and Compose.

Continue Project #1 with PostgreSQL and Docker.

Phase target: finish a working backend project and reach approximately 55–60 DSA problems.

PHASE 3 — DAYS 50–70: GenAI Foundations + RAG

Focus: LLM application engineering and practical RAG systems.

WEEK 8 — LLM Fundamentals

Learn tokens, context windows, temperature, system prompts, structured outputs, hallucinations, model selection and function/tool calling.

Start **Project #2: AI Ticket Classifier**.

Output: category, severity, service, summary and recommended action.

Certification: begin one practical GenAI/AI course or credential if it directly reinforces this learning. Maximum 2–3 hours per week.

WEEK 9 — Embeddings + RAG

Learn embeddings, document ingestion, chunking, vector search, metadata, top-k retrieval and context construction.

Use PostgreSQL + pgvector.

DSA: continue trees, heaps and graph fundamentals.

WEEK 10 — Enterprise Knowledge Assistant

Build an Enterprise RAG Assistant that answers engineering and runbook questions using source-backed retrieval.

Learn reranking, retrieval quality and basic RAG evaluation.

Continue the GenAI credential only if it is supporting practical work.

Phase target: approximately 70–75 quality DSA problems.

PHASE 4 — DAYS 71–91: Tool Calling + Agents + Cloud

Focus: production AI workflows and practical AWS foundations.

WEEK 11 — Tool Calling

Learn how an LLM decides which backend function to call, receives the result and produces an interpreted answer.

Example tools: get_service_health(), get_failed_transactions(), get_recent_deployments(), search_runbooks() and get_previous_incidents().

DSA: graphs, BFS, DFS and revision.

WEEK 12 — Agent Workflows

Learn state, workflows, tool use, retries, guardrails, human-in-the-loop and structured outputs.

Use LangGraph or another workflow framework if useful.

Prioritize one reliable workflow over complicated multi-agent demos.

WEEK 13 — AWS Foundations

Learn IAM, EC2, S3, RDS, Lambda and CloudWatch at a practical level.

Start AWS certification preparation only after practical exposure to these services.

Maximum certification time: 2–3 hours per week.

DSA: heaps, backtracking introduction and mixed revision.

PHASE 5 — DAYS 92–112: Flagship Project + Applications

Focus: build the strongest portfolio project and begin active job applications.

WEEK 14 — Flagship Architecture

Project #3: AI Production Incident Investigator.

Architecture: User → FastAPI → AI Orchestrator → SQL Tool / RAG Tool / API Tool → PostgreSQL / pgvector / Service APIs → LLM → Incident Report.

Design data flow, database structure, APIs and tool interfaces.

WEEK 15 — Investigation Workflow

Implement SQL analysis, evidence gathering, RAG retrieval, deployment checking and incident correlation.

Example request: “Why did payment failures increase between 2–3 PM?”

Target output: Incident, Impact, Likely Root Cause, Evidence, Confidence and Recommended Actions.

Safety: natural-language-to-SQL should use a read-only database, allow-listed tables/columns, validation and query limits.

WEEK 16 — Integration + Deployment

Integrate the workflow end to end.

Add logging, error handling, evaluation, caching where useful and Docker.

Deploy practical components to AWS where appropriate.

Career: begin active applications if you have not already.

PHASE 6 — DAYS 113–120: Interview Sprint + Final Polish

Focus: DSA revision, system design, AI system design and portfolio polish.

WEEK 17 — Interview Mode

DSA: mixed timed practice focused on weak areas.

System design: URL shortener, rate limiter, notification system, file upload service and chat system.

AI system design: RAG architecture, ingestion, chunking, embeddings, retrieval, reranking, caching, latency, cost, security, evaluation and monitoring.

Portfolio: finalize READMEs, architecture diagrams, setup instructions, screenshots/examples and GitHub documentation.

Career: update resume, LinkedIn and continue applications.

Certification Strategy

Maximum certifications during the 120 days: 2.

Certification 1: one practical GenAI/AI credential around Days 50–70.

Certification 2: AWS certification preparation around Days 71–120.

Do not delay projects, DSA or job applications to finish a certification.

If the certification is not helping your project or target role, reduce its priority.

Final DSA Strategy

Target: approximately 100–110 quality problems by Day 120.

Days 1–28: 40–45 problems with strong foundations.

Days 29–49: approximately 55–60 total problems while learning backend.

Days 50–70: approximately 70–75 total problems with trees, heaps and graphs.

Days 71–91: approximately 80–90 total problems with graph practice, backtracking and revision.

Days 92–112: approximately 90–105 total problems with Medium problems and basic DP.

Days 113–120: revision, timed practice and weak-topic repair.

Goal: 100 problems solved well and revisited is better than 200 problems solved and forgotten.

Portfolio by Day 120

Project #1: Production Ticket Management API — Python + FastAPI + PostgreSQL + authentication + Docker.

Project #2: Enterprise RAG Assistant — Python + FastAPI + PostgreSQL + pgvector + embeddings + LLM.

Project #3: AI Production Incident Investigator — Python + FastAPI + PostgreSQL + pgvector + LLM + RAG + tool calling + workflow/agent logic + Docker + AWS.

The third project remains the flagship because it connects enterprise production experience with modern AI application engineering.

Final Career Positioning

Suggested headline: Software Engineer | Python | Backend | GenAI | RAG | LLM Applications.

Career narrative: Enterprise production experience → Python/backend → cloud → GenAI → LLM applications → RAG → tool calling → AI-powered systems.

Do not position yourself as someone who simply collected AI certificates. Position yourself as an engineer who can build production-oriented AI applications.

Final Rule

Do not try to become a deep-learning expert, ML researcher, cloud architect, Kubernetes expert and competitive programmer within 120 days.

Your focused objective: Become employable as a Software / Backend Engineer who can build production-oriented AI applications.

Consistency beats intensity. A realistic 120-day plan that you follow is far more valuable than an aggressive 90-day plan that you abandon halfway.